```
needed <- c("tidyverse","readr","tidytext","textstem","tm","Matrix")
installed <- rownames(installed.packages())
for (pkg in setdiff(needed, installed)) install.packages(pkg)

library(conflicted)
conflict_prefer("filter", "dplyr")
conflict_prefer("lag", "dplyr")

#list and read text files
library(tidyverse)
library(readr)
library(tidytext)
library(dplyr)
library(stringr)
library(textstem)
library(tm)
library(Matrix)
library(RSpectra)
library(ca)
library(ggrepel)
library(caret)

#loading our the data
data <- "C:/Users/Davlatmoh/Downloads/bbc-datasets-main/bbc-datasets-
main/raw/bbc-fulltext"

#getting all our text files inside of each categories(in our case folders)
```

```
files <- list.files(path = data,
                    pattern = "\\.txt$",
                    recursive = TRUE,
                    full.names = TRUE)
#building a table of our files and adding just three columns
articles <- tibble(file = files) %>%
  mutate(
    category = basename(dirname(file)), # here we are specifying folder name
    text = map_chr(file, read_file), # here we are specifying full document text inside of
each categories
    doc_id   = row_number()
  )
articles %>% count(category)#counting how many files are in each category
articles %>% slice_head(n = 3) %>% select(category, text) %>% print()


#Preprocessing steps(changing all characters to lowercase, removing punctuation
marks, numbers, and stop words, and applying lemmatization )
#here we changing characters, removing punctuation and dropping pure-digit numbers
tokens <- articles %>%
  unnest_tokens(word, text) %>%
  filter(!str_detect(word, "^[0-9]+$"))


#Removing English stop words
data("stop_words")
tokens <- tokens %>%
  anti_join(stop_words, by = "word")


#Lemmatize the words
```

```
tokens <- tokens %>%
  mutate(lemma = lemmatize_words(word))

#Term-document counts and filtering step

#Here we are counting term frequencies per each document
term_doc_counts <- tokens %>%
  count(doc_id, lemma, name = "count")

#Computing global term frequencies(counting term frequencies across all documents)
global_counts <- term_doc_counts %>%
  group_by(lemma) %>%
  summarize(total = sum(count), .groups = "drop")

#Filtering out rare terms whose global frequency < 10
common_lemmas <- global_counts %>%
  filter(total >= 5) %>%
  pull(lemma)

filtered_counts <- term_doc_counts %>%
  filter(lemma %in% common_lemmas)

#Building Document-Term Matrix for our further analysis
dtm <- filtered_counts %>%
  cast_sparse(doc_id,lemma, count)
#making sure the correct sparse format for RSpectra
if (!inherits(dtm, "dgCMatrix")) dtm <- as(dtm, "dgCMatrix")
```

```
#making some checking
cat("Number of documents:", length(unique(filtered_counts$doc_id)), "\n")
cat("Vocabulary size (number of terms ≥10):", ncol(dtm), "\n")
cat("DTM dimensions (docs × terms):", dim(dtm)[1], "×", dim(dtm)[2], "\n")

cat("First 20 terms in vocabulary:\n")
print(colnames(dtm)[1:20])

#LSA-RAW & CA-RAW(RAW in our case means that there is no dimentionality
reduction yet)

#we are setting the number of dimensions
k <- 4

#LSA-RAW: truncated SVD of the raw DTM (F data matrix is approximated to U Σ Vᵀ)
lsa_raw <- RSpectra::svds(dtm, k = k)
# coordinates of our document in data matrix: U * diag(d)
docs_lsa  <- lsa_raw$u %*% diag(lsa_raw$d)
#coordinates of our term in data matrix: V * diag(d)
terms_lsa <- lsa_raw$v %*% diag(lsa_raw$d)

#Applying CA-RAW on standardized residuals metrix S
# Build TF–IDF DTM
df_j   <- colSums(dtm > 0)
idf    <- log(nrow(dtm) / df_j)
tfidf  <- Diagonal(x = 1/rowSums(dtm)) %*% dtm %*% Diagonal(x = idf)
```

```
N_t    <- sum(tfidf)
P_t    <- tfidf / N_t
r_t    <- rowSums(P_t); c_t <- colSums(P_t)
Dr_t   <- Diagonal(x = 1/sqrt(r_t))
Dc_t   <- Diagonal(x = 1/sqrt(c_t))
S_t    <- Dr_t %*% (P_t - r_t %*% t(c_t)) %*% Dc_t
#Now we are computing CA coordinates via truncated SVD
k <- 4  # or your preferred k
sv_ca <- svds(S, k = k, nu = k, nv = k)
U_ca      <- sv_ca$u      # (n_docs × k)
Sigma_ca  <- diag(sv_ca$d)# (k × k)
#Principal coordinates (F = U × Σ)
docs_ca <- U_ca %*% Sigma_ca

#printing the singular values of LSA
cat("\nLSA singular values (first k)\n")
print(lsa_raw$d)
#printing the eigenvalues of CA
eigenvalues_ca <- sv_ca$d^2
cat("CA Eigenvalues:\n")
print(eigenvalues_ca[1:4])

# LSA biplot (documents as points, top-20 terms labeled)
df_docs_lsa <- as_tibble(docs_lsa[, 1:2], .name_repair = ~c("Dim1","Dim2")) %>%
  mutate(doc_id = row_number()) %>%   # use row_number() instead of row.names()
  left_join(articles %>% select(doc_id, category), by = "doc_id")
```

```
df_terms_lsa <- as_tibble(terms_lsa[, 1:2], .name_repair = ~c("Dim1","Dim2")) %>%
  mutate(label = colnames(dtm))
p_lsa <- ggplot() +
  geom_point(data = df_docs_lsa,
             aes(Dim1, Dim2, color = category),
             alpha = 0.4, size = 1) +
  geom_text_repel(data = df_terms_lsa %>%
                    slice_max(abs(Dim1) + abs(Dim2), n = 10),
                  aes(Dim1, Dim2, label = label),
                  size = 3) +
  coord_fixed() +
  theme_minimal() +
  labs(title = "LSA-RAW biplot (k=4)", color = "Category")

print(p_lsa)

# CA biplot
# Getting term coordinates for CA in other words principal coordinates
terms_ca <- U_ca %*% Sigma_ca  # term coordinates (principal coords)
#Choosing a subset of terms in other words top 20 contributed terms
term_contrib <- rowSums(terms_ca^2)
top_terms <- order(term_contrib, decreasing = TRUE)[1:19]
# Preparing data frame for terms
df_terms <- tibble(
  term = colnames(dtm)[top_terms],
  Dim1 = terms_ca[top_terms, 1],
  Dim2 = terms_ca[top_terms, 2]
```

```
)
# Preparing data frame for documents
set.seed(123)  # reproducible selection
df_docs <- tibble(
  doc_id = 1:nrow(docs_ca),
  Dim1 = docs_ca[, 1],
  Dim2 = docs_ca[, 2],
  category = articles$category
) %>% slice_sample(n = min(200, nrow(docs_ca)))

# Biplot
ggplot() +
  geom_point(data = df_docs, aes(x = Dim1, y = Dim2, color = category),
             alpha = 0.6, size = 2) +
  geom_text_repel(data = df_terms, aes(x = Dim1, y = Dim2, label = term),
                  size = 4, fontface = "bold", color = "black") +
  theme_minimal(base_size = 14) +
  labs(
    title = "CA Biplot (k=4, Dimensions 1 & 2)",
    x = "Dimension 1",
    y = "Dimension 2",
    color = "Category"
  ) +
  theme(legend.position = "bottom")

#Finding out the MAP via 11-point interpolation for LSA-RAW and CA-RAW
```

```
#varying the number of k-dimensions from 1-20 to find out better performance of the methods

# 80/20 split (you already have this)

#computing MAP

compute_map_db <- function(queries, query_cats, db, db_cats) {

  nq  <- nrow(queries)

  aps <- numeric(nq)

  for (i in seq_len(nq)) {

    dists  <- sqrt(rowSums((db - queries[i, ])^2))

    ord    <- order(dists)

    rel    <- as.integer(db_cats[ord] == query_cats[i])

    cumrel <- cumsum(rel)

    prec   <- cumrel / seq_along(rel)

    total  <- sum(db_cats == query_cats[i], na.rm=TRUE)

    if (total == 0) { aps[i] <- 0; next }

    recalls <- seq(0,1,by=0.1)

    interp  <- sapply(recalls, function(r) {

      min_k <- ceiling(r * total)

      if (min_k==0) return(max(prec, na.rm=TRUE))

      hits <- which(cumrel>=min_k)

      if (!length(hits)) return(0)

      max(prec[hits[1]:length(prec)], na.rm=TRUE)

    })

    aps[i] <- mean(interp)

  }

  mean(aps)

}
```

```
#Computing the MAP of LSA and CA over k dimensions
ks      <- c(4, 6, 10)
reps    <- 10
map_lsa <- matrix(0, nrow = reps, ncol = length(ks))
map_ca  <- matrix(0, nrow = reps, ncol = length(ks))

set.seed(123)
for (r in seq_len(reps)) {
  # 1) Stratified 80/20 split
  train_idx <- articles %>%
    mutate(idx = row_number()) %>%
    group_by(category) %>%
    group_map(~ sample(.x$idx, floor(0.5 * nrow(.x))), .keep = FALSE) %>%
    unlist()
  val_idx   <- setdiff(seq_len(nrow(articles)), train_idx)

  train_dtm  <- dtm[train_idx, ]
  val_dtm    <- dtm[val_idx, ]
  train_cats <- articles$category[train_idx]
  val_cats   <- articles$category[val_idx]

  # 2) Drop any zero-sum terms in training
  keep_cols  <- which(colSums(train_dtm) > 0)
  train_dtm  <- train_dtm[, keep_cols]
  val_dtm    <- val_dtm[, keep_cols]

  # 3) Build CA's S on TF–IDF (instead of raw counts)
```

```
#    3a) Compute TF–IDF on train:
df_j         <- colSums(train_dtm > 0)                  # doc-freq
idf          <- log(nrow(train_dtm) / df_j)             # IDF weights
tfidf_train  <- Diagonal(x = 1/rowSums(train_dtm)) %*%
  train_dtm %*%
  Diagonal(x = idf)                      # TF–IDF matrix

#    3b) Standardized residuals on TF–IDF:
Ntr_tfidf    <- sum(tfidf_train)
P_tfidf      <- tfidf_train / Ntr_tfidf
r_tr         <- rowSums(P_tfidf)
c_tr         <- colSums(P_tfidf)
Dr_tr        <- Diagonal(x = 1 / sqrt(r_tr))
Dc_tr        <- Diagonal(x = 1 / sqrt(c_tr))
S_tr         <- Dr_tr %*% (P_tfidf - r_tr %*% t(c_tr)) %*% Dc_tr

# 4) Loop over k
for (j in seq_along(ks)) {
  k <- ks[j]

  # 4a) LSA-RAW on raw counts
  sv_lsa           <- svds(train_dtm, k = k, nu = k, nv = k)
  docs_lsa_train  <- sv_lsa$u %*% diag(sv_lsa$d)
  docs_lsa_val    <- as.matrix(val_dtm %*% sv_lsa$v)
  map_lsa[r, j]   <- compute_map_db(
    queries    = docs_lsa_val,
    query_cats = val_cats,
```

```
      db         = docs_lsa_train,
      db_cats    = train_cats
    )

    # 4b) CA-RAW on TF–IDF residuals
    sv_ca            <- svds(S_tr, k = k, nu = k, nv = k)
    docs_ca_train   <- sv_ca$u %*% diag(sv_ca$d)
    docs_ca_val     <- t(apply(val_dtm, 1, function(x) {
      prof     <- as.numeric(x) / Ntr_tfidf
      ri       <- sum(prof)
      resid    <- prof - ri * c_tr
      z        <- resid / sqrt(c_tr)
      dr_inv   <- 1 / sqrt(ri)
      as.numeric(dr_inv * (z %*% sv_ca$v))
    }))
    map_ca[r, j]    <- compute_map_db(
      queries    = docs_ca_val,
      query_cats = val_cats,
      db         = docs_ca_train,
      db_cats    = train_cats
    )
  }
}

# 5) Average over the 10 splits and report
mean_lsa <- colMeans(map_lsa)
mean_ca  <- colMeans(map_ca)
```

```
results_df <- tibble(
  k       = ks,
  MAP_LSA = mean_lsa,
  MAP_CA  = mean_ca
)
print(results_df)

#The plot for LSA_RAW and CA
results_df <- tibble(
  k = c(4,6, 10),
  MAP_LSA = c(0.372, 0.349, 0.279),
  MAP_CA = c(0.640, 0.420, 0.380)
)

# reshape data to long format clearly
df_plot <- results_df %>%
  filter(k %in% c(4, 6, 10))

# 2) Reshape to long form
df_long <- df_plot %>%
  pivot_longer(c(MAP_LSA, MAP_CA),
               names_to  = "method",
               values_to = "MAP") %>%
  mutate(method = recode(method,
                         MAP_LSA = "LSA-RAW",
                         MAP_CA  = "CA-RAW"))
```

```
# 3) Plot
ggplot(df_long, aes(x = k, y = MAP,
                    shape    = method,
                    linetype = method,
                    color    = method)) +
  geom_line(linewidth = 0.7) +
  geom_point(size = 4) +
  scale_shape_manual(values = c("LSA-RAW" = 16, "CA-RAW" = 17)) +
  scale_linetype_manual(values = c("LSA-RAW" = "dashed",
                                   "CA-RAW" = "dotted")) +
  scale_color_manual(values = c("LSA-RAW" = "black",
                                "CA-RAW"  = "red")) +
  theme_bw(base_size = 14) +
  labs(
    title = "MAP (Euclidean) vs. k for LSA-RAW (●) and CA-RAW (▲)",
    x     = "k (dimensions)",
    y     = "Mean Average Precision"
  ) +
  theme(
    legend.position      = "bottom",
    legend.title         = element_blank(),
    panel.grid.minor     = element_blank(),
    panel.grid.major     = element_blank()
  )

#using weighting components to our LSA in order to improve the MAP
```

```
compute_map <- function(coords, categories) {
  # in LOOCV the "db" is the same as the "queries"
  compute_map_db(
    queries    = coords,
    query_cats = categories,
    db         = coords,
    db_cats    = categories
  )
}
k      <- 4
alphas <- seq(0.1, 1.9, by = 0.1)
cats   <- articles$category

#truncated SVD of the DTM
k <- 4  # or your preferred k
sv_ca <- svds(S, k = k, nu = k, nv = k)
U_ca      <- sv_ca$u      # (n_docs × k)
Sigma_ca  <- diag(sv_ca$d)# (k × k)
#Principal coordinates (F = U × Σ)
docs_ca <- U_ca %*% Sigma_ca



#Sweep α and compute the MAP after weighting component
map_alpha <- map_dbl(alphas, function(alpha) {
  docs_lsa_alpha <- sweep(U_k, 2, D_k^alpha, FUN = "*")
  compute_map(docs_lsa_alpha, cats)
})
```

```
#Results of our LSA MAP after applying alpha
df_a <- tibble(alpha = alphas, LSA_MAP = map_alpha)
print(df_a)

#ploting results
ggplot(df_a, aes(x = alpha, y = LSA_MAP)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  scale_x_continuous(breaks = alphas) +
  theme_minimal(base_size = 14) +
  labs(
    title = sprintf("LOOCV MAP vs. α for LSA-RAW (k = %d)", k),
    x     = expression(alpha),
    y     = "Mean Average Precision"
  )

#here we are looking at the map performance of 4 versions of LSA
k    <- 4
cats <- articles$category

#Defining the four LSA versions
wt_none <- function(m) m

wt_nrow_l1 <- function(m) {
  div <- rowSums(m)
  Diagonal(x = 1/div) %*% m
}
```

```
wt_nrow_l2 <- function(m) {
  div <- sqrt(rowSums(m^2))
  Diagonal(x = 1/div) %*% m
}

df_j  <- colSums(dtm > 0)
idf_j <- log(nrow(dtm) / df_j)

wt_tfidf <- function(m) {
  tf <- Diagonal(x = 1/rowSums(m)) %*% m
  tf %*% Diagonal(x = idf_j)
}

schemes <- list(
  LSA_RAW     = wt_none,
  LSA_NROW_L1 = wt_nrow_l1,
  LSA_NROW_L2 = wt_nrow_l2,
  LSA_TFIDF   = wt_tfidf
)

# determining the map for each 4 versions of LSA at k =4
results <- imap_dfr(schemes, function(wt_fun, name) {
  M    <- wt_fun(dtm)                    # apply weighting
  sv   <- svds(M, k = k, nu = k, nv = k)  # truncated SVD
  docs <- sweep(sv$u, 2, sv$d, FUN = "*") # U Σ
  MAP  <- compute_map(docs, cats)        # 11-point MAP
```

```
  tibble(method = name, MAP = MAP)
})
#results
print(results)


#Graphing the four MAP values of LSA versions
results %>%
  mutate(method = factor(method, levels =
c("LSA_RAW","LSA_NROW_L1","LSA_NROW_L2","LSA_TFIDF"),
                       labels = c("RAW","LSA–NROW-L1","LSA–NROW-L2","LSA–TFIDF")))
%>%
  ggplot(aes(x = method, y = MAP, fill = method)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_text(aes(label = sprintf("%.3f", MAP)), vjust = -0.5, size = 5) +
  scale_fill_brewer(palette = "Set2") +
  coord_cartesian(ylim = c(0, max(results$MAP) + 0.05)) +
  theme_minimal(base_size = 14) +
  labs(
    title = sprintf("MAP where k=%d for 4 LSA Versions", k),
    x     = NULL,
    y     = "Mean Average Precision"
  )


#Looking for teh MAP performance of CA based on weighting
ks      <- c(4)
alphas  <- seq(1, 2, by = 0.1)
cats    <- articles$category
```

```
# Precompute SVD of S for each k (so we don't redo svds() inside the alpha loop)
svd_list <- map(ks, function(k) {
  sv_ca <- svds(S, k = k, nu = k, nv = k)
})
names(svd_list) <- as.character(ks)
#Sweep alpha for each k and compute MAP
df_ca <- map_dfr(ks, function(k) {
  sv_ca <- svd_list[[as.character(k)]]
  U_ca  <- sv_ca$u
  d_ca  <- sv_ca$d
  map_alpha <- map_dbl(alphas, function(alpha) {
    docs_ca_alpha <- sweep(U_ca, 2, d_ca^alpha, FUN = "*")
    compute_map(docs_ca_alpha, cats)
  })
  tibble(k = k, alpha = alphas, CA_MAP = map_alpha)
})

#Print the results
print(df_ca, n = nrow(df_ca))

# Plot MAP
ggplot(df_ca, aes(x = alpha, y = CA_MAP)) +
  geom_line(color = "#33a02c", linewidth = 1) +
  geom_point(color = "#33a02c", size = 3) +
  theme_minimal(base_size = 14) +
  labs(
    title = sprintf("MAP vs. α for CA-RAW (k = %d)", k),
```

```
    x     = expression(alpha),
    y     = "Mean Average Precision"
  )

#Ploting the MAP of CA after aplying alpha
df_ca <- tibble(
  k      = 4,
  alpha  = seq(1.0, 2.0, by = 0.1),
  CA_MAO = c(
    rep(640.00, 4),    # α = 1.0, 1.1, 1.2, 1.3
    rep(638.42, 6),    # α = 1.4, 1.5, 1.6, 1.7, 1.8, 1.9
    636.63             # α = 2.0
  )
)
ggplot(df_ca, aes(x = alpha, y = CA_MAO, color = factor(k), group = factor(k))) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_color_brewer("k", palette = "Set1") +
  theme_minimal(base_size = 14) +
  labs(
    title = "CA-RAW: CA_MAO vs. α for k = 4",
    x     = expression(alpha),
    y     = "CA_MAO"
  )
```